

GROWMORE GROUP OF INSTITUTE

UNIT - 4

File management: File manager, Access method, Directory structure

What is File System?

A file is a collection of correlated information which is recorded on secondary or non-volatile (ROM) storage like magnetic disks, optical disks, and tapes. It is a method of data collection that is used as a medium for giving input and receiving output from that program.

In general, a file is a sequence of bits, bytes, or records whose meaning is defined by the file creator and user. Every File has a logical location where they are located for storage and retrieval.

Objective of File management System

Here are the main objectives of the file management system:

- It provides I/O support for a variety of storage device types.
- Minimizes the chances of lost or destroyed data
- It provides I/O support for multiple users in a multiuser systems environment.

Properties of a File System

Here, are important properties of a file system:

- Files have names and are associated with access permission that permits controlled sharing.
- Files could be arranged or more complex structures to reflect the relationship between them.

File structure

A File Structure needs to be predefined format in such a way that an operating system understands. It has an exclusively defined structure, which is based on its type.

Three types of files structure in OS:

- **A text file:** It is a series of characters that is organized in lines.
- **An object file:** It is a series of bytes that is organized into blocks.
- **A source file:** It is a series of functions and processes.

File Attributes

A file has a name and data. Moreover, it also stores meta information like file creation date and time, current size, last modified date, etc. All this information is called the attributes of a file system.

Here, are some important File attributes used in OS:

- **Name:** It is the only information stored in a human-readable form.
- **Identifier:** Every file is identified by a unique tag number within a file system known as an identifier.
- **Location:** Points to file location on device.
- **Type:** This attribute is required for systems that support various types of files.
- **Size.** Attribute used to display the current file size.
- **Protection.** This attribute assigns and controls the access rights of reading, writing, and executing the file.
- **Time, date and security:** It is used for protection, security, and also used for monitoring

File Type

It refers to the ability of the operating system to differentiate various types of files like text files, binary, and source files. However, Operating systems like MS_DOS and UNIX has the following type of files:

Character Special File

It is a hardware file that reads or writes data character by character, like mouse, printer, and more.

Ordinary files

- These types of files stores user information.
- It may be text, executable programs, and databases.
- It allows the user to perform operations like add, delete, and modify.

Directory Files

- Directory contains files and other related information about those files. Its basically a folder to hold and organize multiple files.

Special Files

- These files are also called device files. It represents physical devices like printers, disks, networks, flash drive, etc.

Functions of File

- Create file, find space on disk, and make an entry in the directory.
- Write to file, requires positioning within the file.
- Delete directory entry, regain disk space.
- Reposition: move read/write position.

Commonly used terms in File systems

Field:

This element stores a single value, which can be static or variable length.

DATABASE:

Collection of related data is called a database. Relationships among elements of data are explicit.

FILES:

Files is the collection of similar record which is treated as a single entity.

RECORD:

A Record type is a complex data type that allows the programmer to create a new data type with the desired column structure. Its groups one or more columns to form a new data type. These columns will have their own names and data type.

File Access Methods

File access is a process that determines the way that files are accessed and read into memory. Generally, a single access method is always supported by operating systems.

Three file access methods are:

- Sequential access
- Direct random access
- Index sequential access

Sequential Access

In this type of file access method, records are accessed in a certain pre-defined sequence. In the sequential access method, information stored in the file is also processed one by one. Most compilers access files using this access method.

Random Access

The random access method is also called direct random access. This method allow accessing the record directly. Each record has its own address on which can be directly accessed for reading and writing.

Sequential Access

This type of accessing method is based on simple sequential access. In this access method, an index is built for every file, with a direct pointer to different memory blocks. In this method, the Index is searched sequentially, and its pointer can access the file directly.

Space Allocation

In the Operating system, files are always allocated disk spaces.

Three types of space allocation methods are:

- Linked Allocation
- Indexed Allocation
- Contiguous Allocation

Contiguous Allocation

In this method,

- Every file users a contiguous address space on memory.
- Here, the OS assigns disk address is in linear order.
- In the contiguous allocation method, external fragmentation is the biggest issue.

Linked Allocation

In this method,

- Every file includes a list of links.
- The directory contains a link or pointer in the first block of a file.
- With this method, there is no external fragmentation
- This File allocation method is used for sequential access files.
- This method is not ideal for a direct access file.

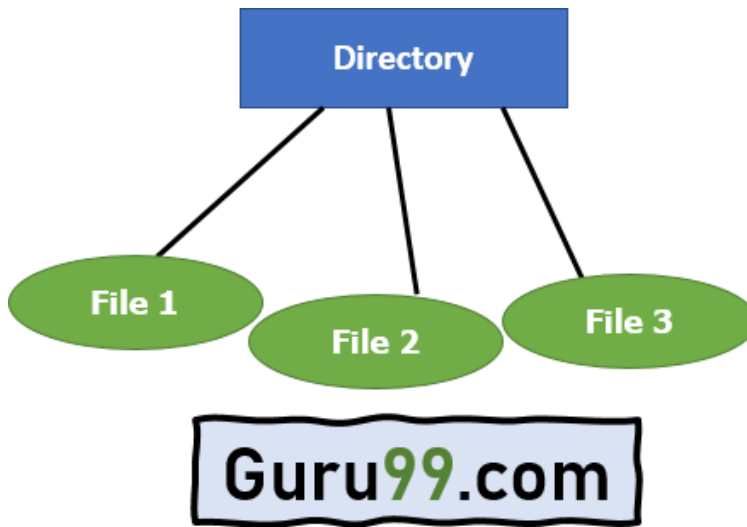
Indexed Allocation

In this method,

- Directory comprises the addresses of index blocks of the specific files.
- An index block is created, having all the pointers for specific files.
- All files should have individual index blocks to store the addresses for disk space.

File Directories

A single directory may or may not contain multiple files. It can also have sub-directories inside the main directory. Information about files is maintained by Directories. In Windows OS, it is called folders.



Single Level Directory

Following is the information which is maintained in a directory:

- **Name** The name which is displayed to the user.
- **Type**: Type of the directory.
- **Position**: Current next-read/write pointers.
- **Location**: Location on the device where the file header is stored.
- **Size**: Number of bytes, block, and words in the file.
- **Protection**: Access control on read/write/execute/delete.
- **Usage**: Time of creation, access, modification

File types- name, extension

File Type	Usual extension	Function
Executable	exe, com, bin or none	ready-to-run machine- language program
Object	obj, o	compiled, machine language, not linked
Source code	c. p, pas, 177, asm, a	source code in various languages
Batch	bat, sh	Series of commands to be executed
Text	txt, doc	textual data documents
Word processor	doc, docs, tex, rrf, etc.	various word-processor formats
Library	lib, h	libraries of routines
Archive	arc, zip, tar	related files grouped into one file, sometimes compressed.

What Is UNIX?

- UNIX is a computer operating system.
- An operating system is the program that controls all the other parts of a computer system, both the hardware and the software. It allocates the computer's resources and schedules tasks. It allows you to make use of the facilities provided by the system. Every computer requires an operating system.
- UNIX is a multi-user, multi-tasking operating system. Multiple users may have multiple tasks running simultaneously. This is very different from PC operating systems such as MS-DOS or MS-Windows (which allows multiple tasks to be carried out simultaneously but not multiple users).
- UNIX is a machine independent operating system. Not specific to just one type of computer hardware. Designed from the beginning to be independent of the computer hardware.
- UNIX is a software development environment. Was born in and designed to function within this type of environment.

Why UNIX?

- Hardware independence
 - operating system code is written in C language rather than a specific assembly language
 - operating system software can be easily moved from one hardware system to another
 - UNIX applications can be easily moved to other UNIX machines. Porting is usually as simple as transfer of the source and a recompile
- Productive environment for software development
 - rich set of tools
 - versatile command language
- Distributed processing and multi-tasking

Features of UNIX

Unix is an operating system, so it has all the features that the OS must-have. UNIX also looks at a few things in a different way than other OS. Features of UNIX are listed below :

1. Multiuser System :

Unix provides multiple programs to run and compete for the attention of the CPU. This happens in 2 ways :

- Multiple users running multiple jobs
- Single user running multiple jobs

2. Multitask System :

A single user may run multiple tasks concurrently. Example : Editing a file, printing another on the printer & sending email to a person, and browsing the net too at the same time. The Kernel is designed to handle user's multiple needs.

The important thing here is that only one job can be seen running in the foreground, the rest all seems to run in the background. Users can switch between them, terminate/suspend any of the jobs.

3. The Building-Block Approach :

The Unix developers thought about keeping small commands for every kind of work. So Unix has so many commands, each of which performs one simple job only. You can use 2 commands by using pipes ('|'). Example : \$ ls | wc Here, | (pipe) connects 2 commands to create a pipeline. This command counts the number of files in the directory.

4. Pattern Matching :

Unix provides very sophisticated pattern matching features. The meta-char '*' is a special character used by the system to match a number of file names. There are several other meta-char in UNIX. The matching is not confined to only filename. Advanced tools use a regular expression that is framed with the characters from this set.

5. Programming Facility :

Unix provides shell which is also a programming language designed for programmers, not for casual end-users. It has all the control structures, loops, and variables required for programming purposes.

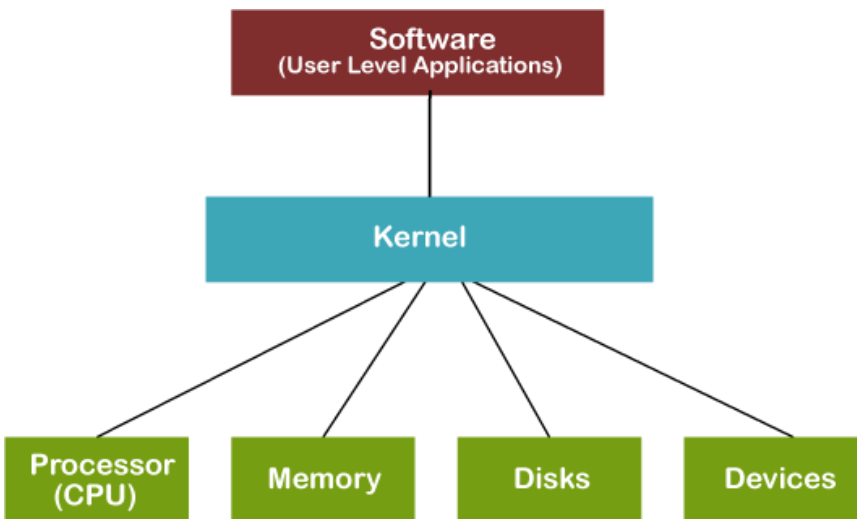
6. Documentation :

It has a 'man' command that stands for the manual, which is the most important reference for any commands and their configuration files. Apart from the online documentation, there is a vast amount of resources available on the Internet. If you're stuck with a problem, there are various UNIX newsgroups where you can post your concerns..

What is Kernel?

In computer science, *Kernel is a computer program that is a core or heart of an operating system.* Before discussing kernel in detail, let's first understand its basic, i.e., Operating system in a computer.

What is Kernel in Operating System?



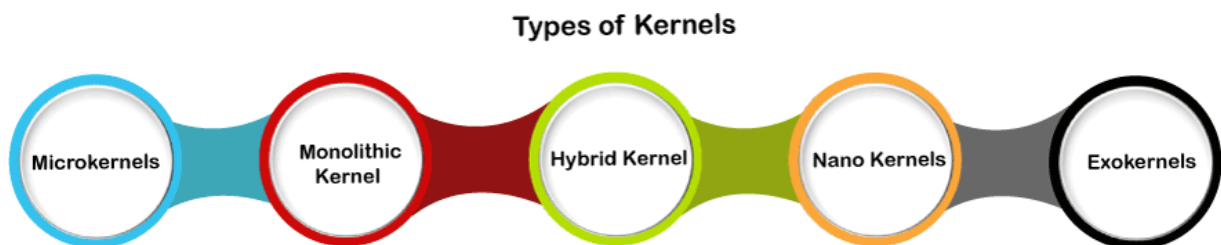
- As discussed above, Kernel is the core part of an OS(Operating system); hence it has full control over everything in the system. Each operation of hardware and software is managed and administrated by the kernel.

- It acts as a bridge between applications and data processing done at the hardware level. It is the central component of an OS.
- It is the part of the OS that always resides in computer memory and enables the communication between software and hardware components.
- It manages other tasks also such as **memory management, task management, and disk management**.
- Other application programs such as browser, word processor, audio & video player use separate memory space known as **user-space**.

Functions of a Kernel

- Device Management
- Memory Management
- Resource Management
- Accessing Computer Resources
- Random-Access Memory

Types of Kernel



1. Monolithic Kernels

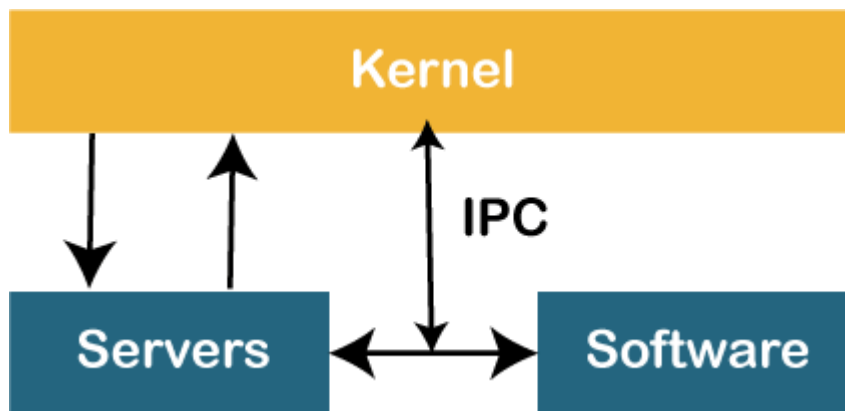
In a monolithic kernel, the **same memory space is used to implement user services and kernel services**.

It means, in this type of kernel, there is no different memory used for user services and kernel services.

As it uses the same memory space, the size of the kernel increases, increasing the overall size of the OS.

Examples of Monolithic Kernels are Unix, Linux, Open VMS, XTS-400, etc.

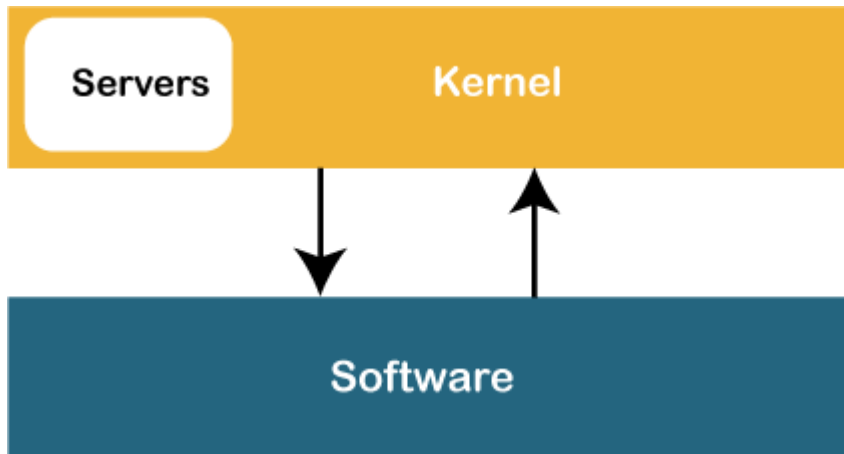
2. Microkernel



A microkernel is also referred to as μK , and it is different from a traditional kernel or Monolithic Kernel. In this, *user services and kernel services are implemented into two different address spaces: user space and kernel space.*

Examples of Microkernel are L4, AmigaOS, Minix, K42, etc.

3. Hybrid Kernel



Hybrid kernels are also known as **modular kernels**, and it is the combination of both Monolithic and Microkernels. It takes advantage of the speed of monolithic kernels and the modularity of microkernels.

Examples of Hybrid Kernel are Windows NT, Netware, BeOS, etc.

4. Nanokernel

As the name suggests, *in Nanokernel, the complete code of the kernel is very small, which means the code executing in the privileged mode of the hardware is very small.* Here the term nano defines a kernel that supports a nanosecond clock resolution.

Examples of Nanokernel are EROS etc.

5. Exokernel

Exokernel is still developing and is the experimental approach for designing OS.

This type of kernel is different from other kernels as in this; resource protection is kept separated from management, which allows us to perform application-specific customization.

Basic Commands:

1. pwd command

Use the **pwd** command to find out the path of the current working directory (folder) you're in. The command will return an absolute (full) path, which is basically a path of all the

directories that starts with a forward slash (/). An example of an absolute path is **/home/username**.

2. cd command

To navigate through the Linux files and directories, use the **cd** command. It requires either the full path or the name of the directory, depending on the current working directory that you're in.

Let's say you're in **/home/username/Documents** and you want to go to **Photos**, a subdirectory of **Documents**.

There are some shortcuts to help you navigate quickly:

- **cd ..** (with two dots) to move one directory up
- **cd** to go straight to the home folder
- **cd-** (with a hyphen) to move to your previous directory

3. ls command

The **ls** command is used to view the contents of a directory. By default, this command will display the contents of your current working directory.

If you want to see the content of other directories, type **ls** and then the directory's path. For example, enter **ls /home/username/Documents** to view the content of **Documents**.

There are variations you can use with the **ls** command:

- **ls -R** will list all the files in the sub-directories as well
- **ls -a** will show the hidden files
- **ls -al** will list the files and directories with detailed information like the permissions, size, owner, etc.

4. cat command

cat (short for concatenate) is one of the most frequently used commands in Linux. It is used to list the contents of a file on the standard output (sdout). To run this command, type **cat** followed by the file's name and its extension. For instance: **cat file.txt**.

Here are other ways to use the **cat** command:

- **cat > filename** creates a new file
- **cat filename1 filename2>filename3** joins two files (1 and 2) and stores the output of them in a new file (3)
- to convert a file to upper or lower case use, **cat filename | tr a-z A-Z >output.txt**

5. cp command

Use the **cp** command to copy files from the current directory to a different directory. For instance, the command **cp scenery.jpg /home/username/Pictures** would create a copy of **scenery.jpg** (from your current directory) into the **Pictures** directory.

6. mv command

The primary use of the **mv** command is to move files, although it can also be used to rename files.

The arguments in **mv** are similar to the **cp** command. You need to type **mv**, the file's name, and the destination's directory. For example: **mv file.txt /home/username/Documents**.

To rename files, the Linux command is **mv oldname.ext newname.ext**

7. mkdir command

Use **mkdir** command to make a new directory — if you type **mkdir Music** it will create a directory called **Music**.

There are extra **mkdir** commands as well:

- To generate a new directory inside another directory, use this Linux basic command **mkdir Music/Newfile**
- use the **p** (parents) option to create a directory in between two existing directories. For example, **mkdir -p Music/2020/Newfile** will create the new “2020” file.

8. rmdir command

If you need to delete a directory, use the **rmdir** command. However, **rmdir** only allows you to delete empty directories.

9. rm command

The **rm** command is used to delete directories and the contents within them. If you only want to delete the directory — as an alternative to **rmdir**

10. grep command

Another basic Linux command that is undoubtedly helpful for everyday use is **grep**. It lets you search through all the text in a given file.

To illustrate, **grep blue notepad.txt** will search for the word blue in the notepad file. Lines that contain the searched word will be displayed fully.

11. diff command

Short for difference, the **diff** command compares the contents of two files line by line. After analyzing the files, it will output the lines that do not match. Programmers often use this

command when they need to make program alterations instead of rewriting the entire source code.

The simplest form of this command is **diff file1.ext file2.ext**

12. chmod command

chmod is another Linux command, used to change the read, write, and execute permissions of files and directories. As this command is rather complicated, you can read [the full tutorial](#) in order to execute it properly.

13. man command

Confused about the function of certain Linux commands? Don't worry, you can easily learn how to use them right from Linux's shell by using the **man** command. For instance, entering **man tail** will show the manual instruction of the tail command.

14. WC command

The wc command in UNIX is **a command line utility for printing newline, word and byte counts for files**. It can return the number of lines in a file, the number of characters in a file and the number of words in a file. It can also be combine with pipes for general counting operations.

15. wildcards

A wildcard in Linux means it might be a symbol or set of symbols representing other characters. It is generally used in substituting any string or a character. Wildcards are mainly used to increase the efficiency and flexibility of searches in Linux.

Process Management commands:

1. ps command

ps command is short for 'Process Status'. It displays the currently-running processes. However, unlike the top command, the output generated is not in realtime.

\$ ps

```
root@localhost:~# ps
  PID TTY          TIME CMD
  84942 pts/0    00:00:00 bash
  85053 pts/0    00:00:00 ps
root@localhost:~#
```

The terminology is as follows :

PID	process ID
TTY	terminal type
TIME	total time the process has been running
CMD	name of the command that launches the process

2. Kill command

[kill command](#) sends a signal to the process.

There are different types of signals that you can send. However, the most common one is ‘kill -9’ which is ‘**SIGKILL**’.

You can list all the signals using:

\$ kill -L

```
LinuxForDevices> kill -l
 1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL      5) SIGTRAP
 6) SIGABRT     7) SIGBUS     8) SIGFPE      9) SIGKILL     10) SIGUSR1
11) SIGSEGV    12) SIGUSR2   13) SIGPIPE    14) SIGALRM    15) SIGTERM
16) SIGSTKFLT 17) SIGCHLD   18) SIGCONT    19) SIGSTOP    20) SIGTSTP
21) SIGTTIN    22) SIGTTOU   23) SIGURG     24) SIGXCPU    25) SIGXFSZ
26) SIGVTALRM 27) SIGPROF   28) SIGWINCH   29) SIGIO       30) SIGPWR
31) SIGSYS     34) SIGRTMIN  35) SIGRTMIN+1 36) SIGRTMIN+2 37) SIGRTMIN+3
38) SIGRTMIN+4 39) SIGRTMIN+5 40) SIGRTMIN+6 41) SIGRTMIN+7 42) SIGRTMIN+8
43) SIGRTMIN+9 44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9 56) SIGRTMAX-8 57) SIGRTMAX-7
58) SIGRTMAX-6 59) SIGRTMAX-5 60) SIGRTMAX-4 61) SIGRTMAX-3 62) SIGRTMAX-2
63) SIGRTMAX-1 64) SIGRTMAX
```

3. Nice command

nice command in Linux helps in execution of a program/process with modified scheduling priority. It launches a process with a user-defined scheduling priority. In this, if we give a process a higher priority, then Kernel will allocate more CPU time to that process.

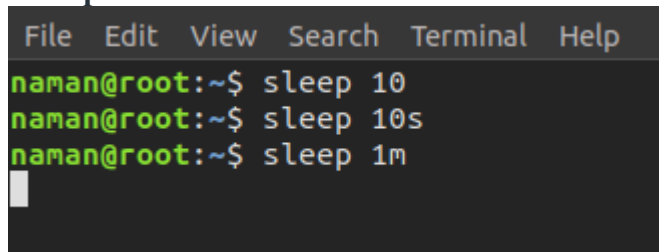
3.sleep command

sleep command is used to create a dummy job. A dummy job helps in delaying the execution. It takes time in seconds by default but a small suffix(s, m, h, d) can be added at the end to convert it into any other format. This command pauses the execution for an amount of time which is defined by *NUMBER*.

Syntax:

sleep NUMBER[SUFFIX]...

Example:

A terminal window with a dark background and light-colored text. The menu bar at the top shows 'File', 'Edit', 'View', 'Search', 'Terminal', and 'Help'. The prompt is 'naman@root:~\$'. Three lines of commands are entered: 'sleep 10', 'sleep 10s', and 'sleep 1m'. Each line is followed by a new prompt 'naman@root:~\$'. A cursor is visible on the line following the last command.

```
File Edit View Search Terminal Help
naman@root:~$ sleep 10
naman@root:~$ sleep 10s
naman@root:~$ sleep 1m
naman@root:~$
```

4. Batch command

batch command is used to read commands from standard input or a specified file and execute them when system load levels permit.

5.at command

at command is a command-line utility that is used to schedule a command to be executed at a particular time in the future. Jobs created with **at command** are executed only once. The **at command** can be used to execute any program or mail at any time in the future. It executes commands at a particular time and accepts times of the form HH:MM to run a job at a specific time of day. The following expression like noon, midnight, teatime, tomorrow, next week, next Monday, etc. could be used with **at command** to schedule a job.

Syntax:

at [OPTION...] runtime